Ontology modifying :

Ontology Extension

Creation of New Classes

The ontology was extended by creating two additional classes:

- `Person`
- `Student`

The `Student` class was modeled as a subclass of `Person`.

Additionally, a new organizational subclass named `University` was created under the `Organization` class.

The resulting hierarchy became:

```
Thing
 ├── GeoEntity
 ├── Organization
 │      ├── Company
 │      ├── Non-profit Organization
 │      └── University
 └── Person
        └── Student
```

Creation of Individuals

Several individuals were added to populate the ontology and support semantic querying and reasoning.

Student Individuals

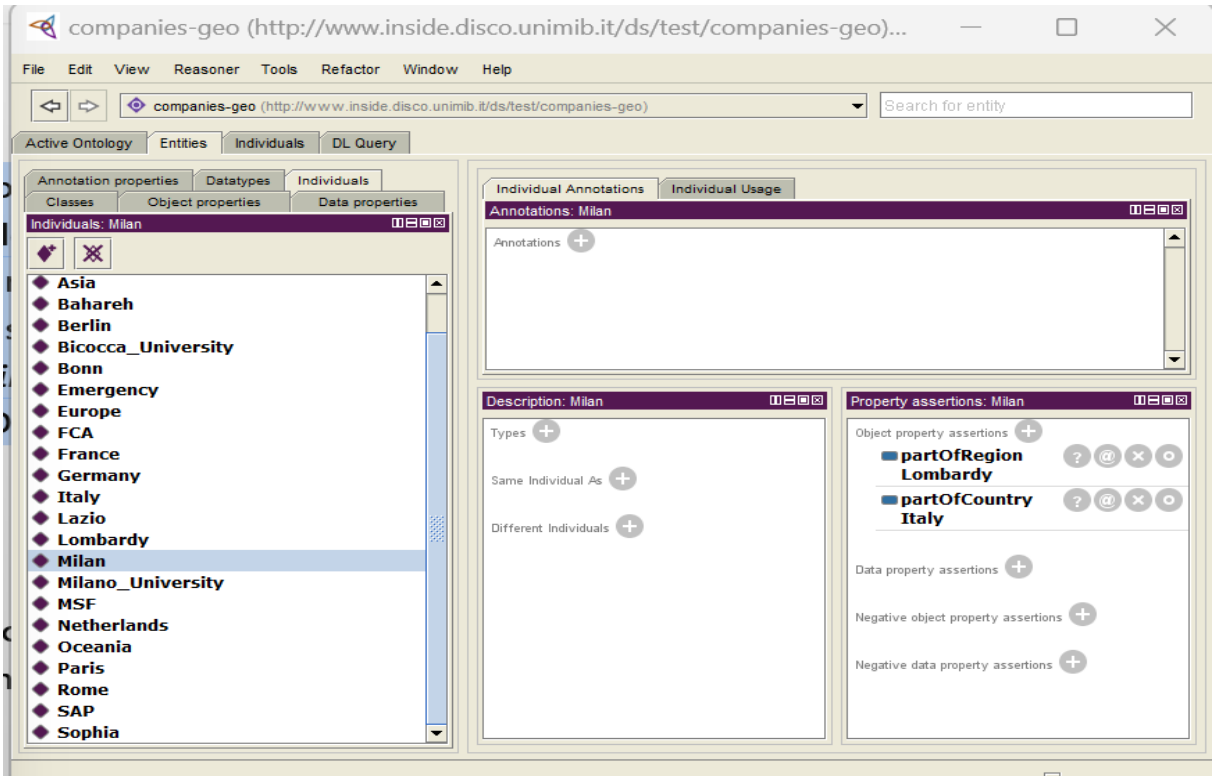
- `Bahareh`
- `Sophia`

University Individuals

- `Milano_University`
- `Bicocca_University`

City Individuals

- `Milan`
- `Rome`



Object Properties

Several semantic object properties were created to model relationships between ontology entities.

friendOf

Represents friendship relations between people.

- Domain: `Person`
- Range: `Person`
- Characteristic: `Symmetric`

The symmetric characteristic allows the reasoner to infer reciprocal friendship relations automatically.

Example triple:

```
Bahareh friendOf Sophia
```

---

hasBirthplace

Represents the birthplace of a person.

- Domain: `Person`
- Range: `City`

Example triples:

```
Bahareh hasBirthplace Milan
Sophia hasBirthplace Rome
```

---

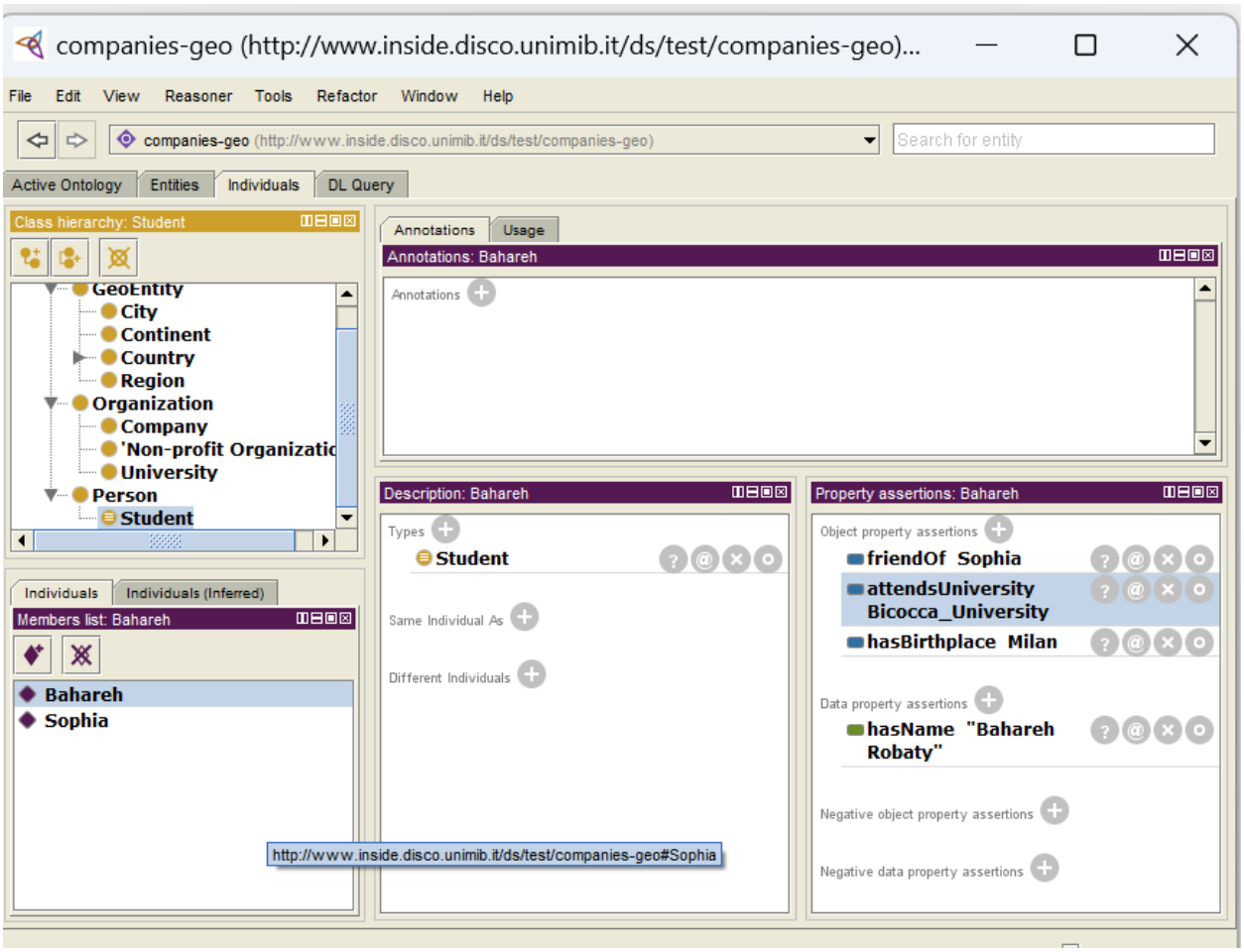
attendsUniversity

Represents the university attended by a student.

- Domain: `Student`
- Range: `University`

Example triples:

```
Bahareh attendsUniversity Milano_University
Sophia attendsUniversity Bicocca_University
```



Data Properties

Two data properties were additionally defined.

hasName

Stores textual names for people.

- Domain: `Person`
- Range: `xsd:string`

Example triples:

```
Bahareh hasName "Bahareh Robaty"
Sophia hasName "Sophia Jackson"
```

---

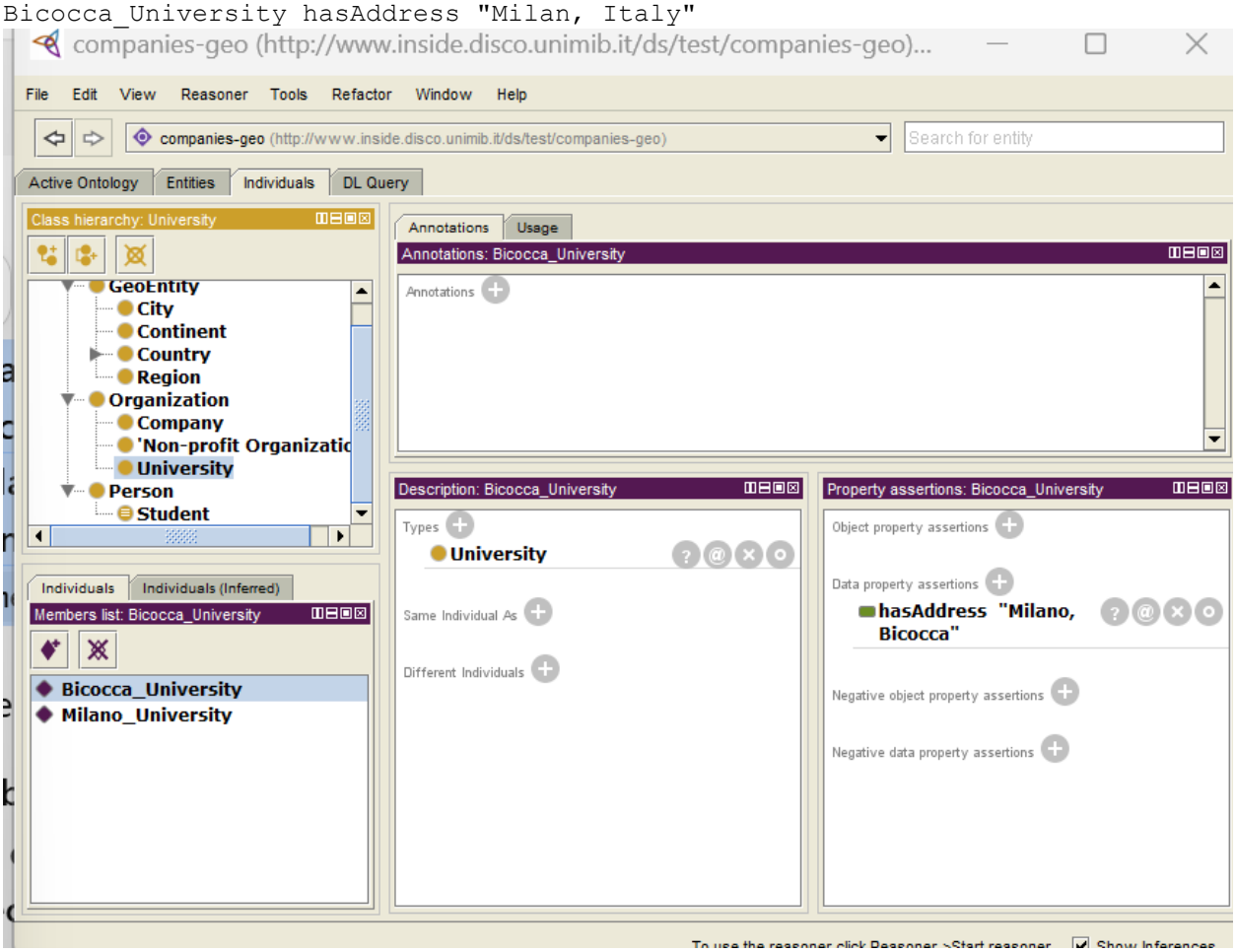
hasAddress

Stores textual addresses for universities.

- Domain: `University`
- Range: `xsd:string`

Example triples:

```
Milano_University hasAddress "Milan, Italy"
Bicocca_University hasAddress "Milan, Italy"
```



RDF Triple Modeling

The ontology relationships were modeled using RDF-style triples in the form:

```
Subject — Predicate — Object
```

Examples include:

```
Bahareh attendsUniversity Milano_University
Sophia hasBirthplace Rome
Bahareh friendOf Sophia
```

These triples collectively formed the semantic knowledge graph of the ontology.

Semantic Validation and Ontology Reasoning

The HermiT reasoner was used to validate ontology consistency and infer implicit semantic knowledge.

Initially, an inconsistency was detected when the object `America` was assigned as the birthplace of a person while the `hasBirthplace` property required the range to be a `City`. Since `America` was classified as a `Continent`, the ontology became inconsistent.

The object property:

```
hasBirthplace
```

was defined with:

- Domain: `Person`
- Range: `City`

An inconsistency occurred when the following assertion was added:

```
Sophia hasBirthplace America
```

However, within the ontology:

```
America rdf:type Continent
```

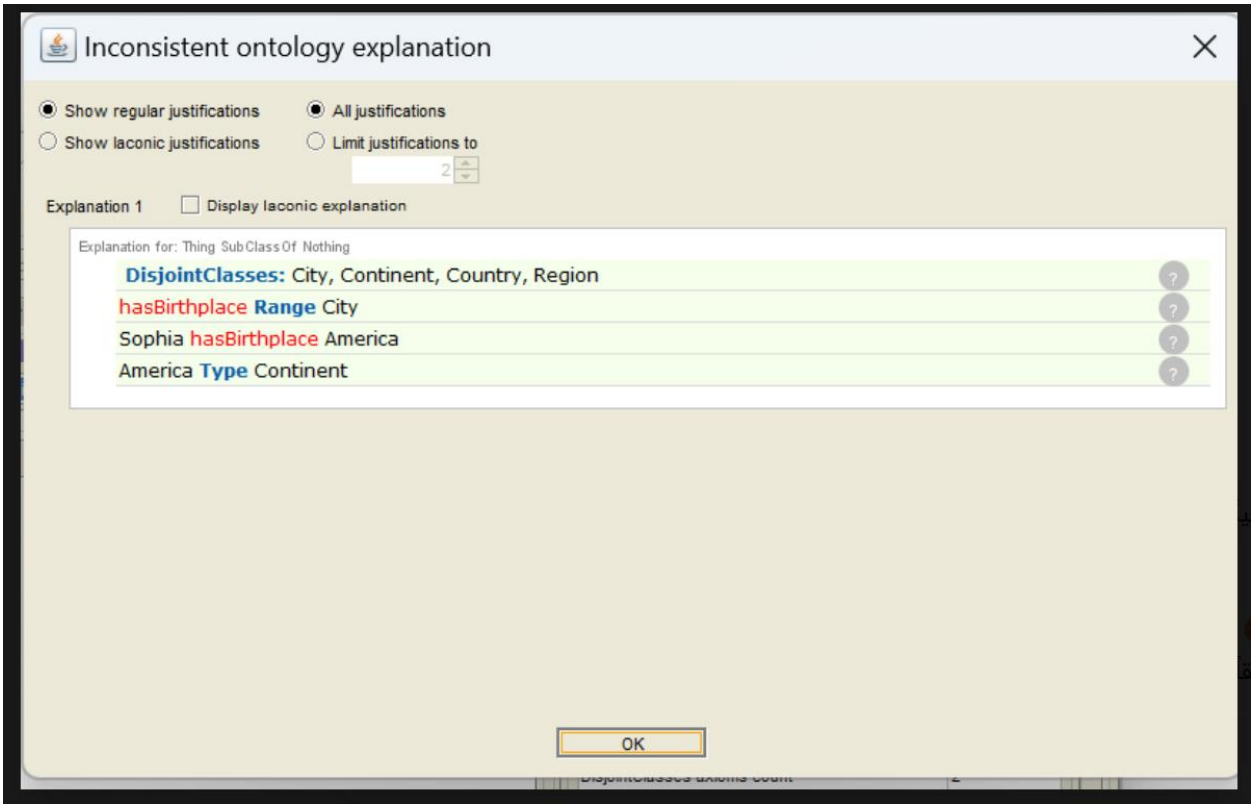
while the property `hasBirthplace` required the object to belong to the `City` class.

Additionally, the ontology defined the following disjoint class restriction:

```
City disjointWith Continent
```

As a result, the reasoner detected a logical contradiction and marked the ontology as inconsistent.

This example demonstrated how range restrictions and disjoint class constraints are used for semantic validation.



Another inconsistency occurred when the individual `Sophia` was simultaneously classified as both a `Student` and a `University`. This violated semantic modeling constraints because those classes represent fundamentally different entity types.

The inconsistency was resolved by removing the incorrect `University` type assertion from `Sophia`.

Inference Discussion

because the `friendOf` property was configured as symmetric, the reasoner automatically inferred reciprocal friendship relations.

For example:

```
Bahareh friendOf Sophia
```

allowed the reasoner to infer:

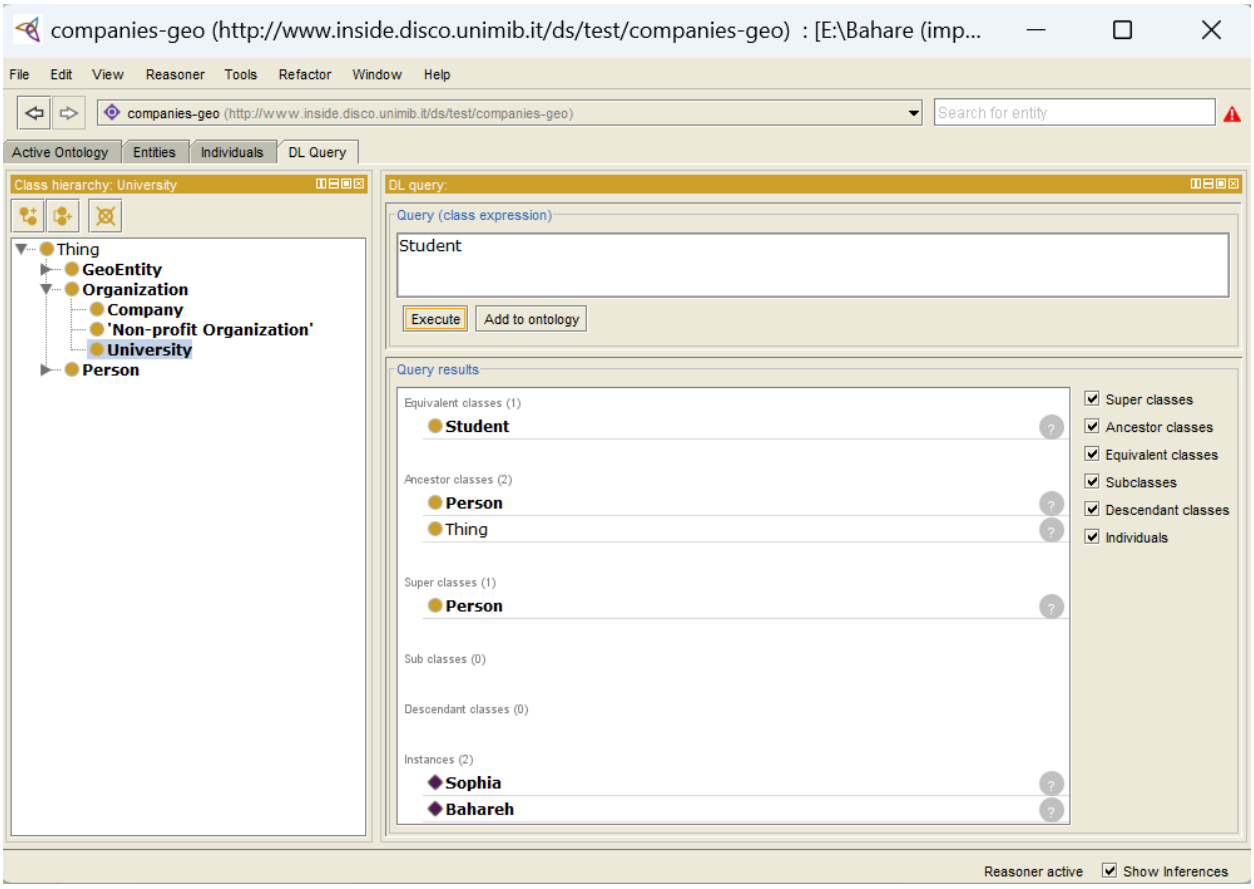
```
Sophia friendOf Bahareh
```

These results demonstrate the power of semantic reasoning in ontology-based systems.

Querying and Validation

The ontology was queried using DL Query in Protégé to validate semantic relationships and inferred knowledge.

Several example queries were executed.



Query 1

```
Student
```

Result:

- Bahareh
- Sophia

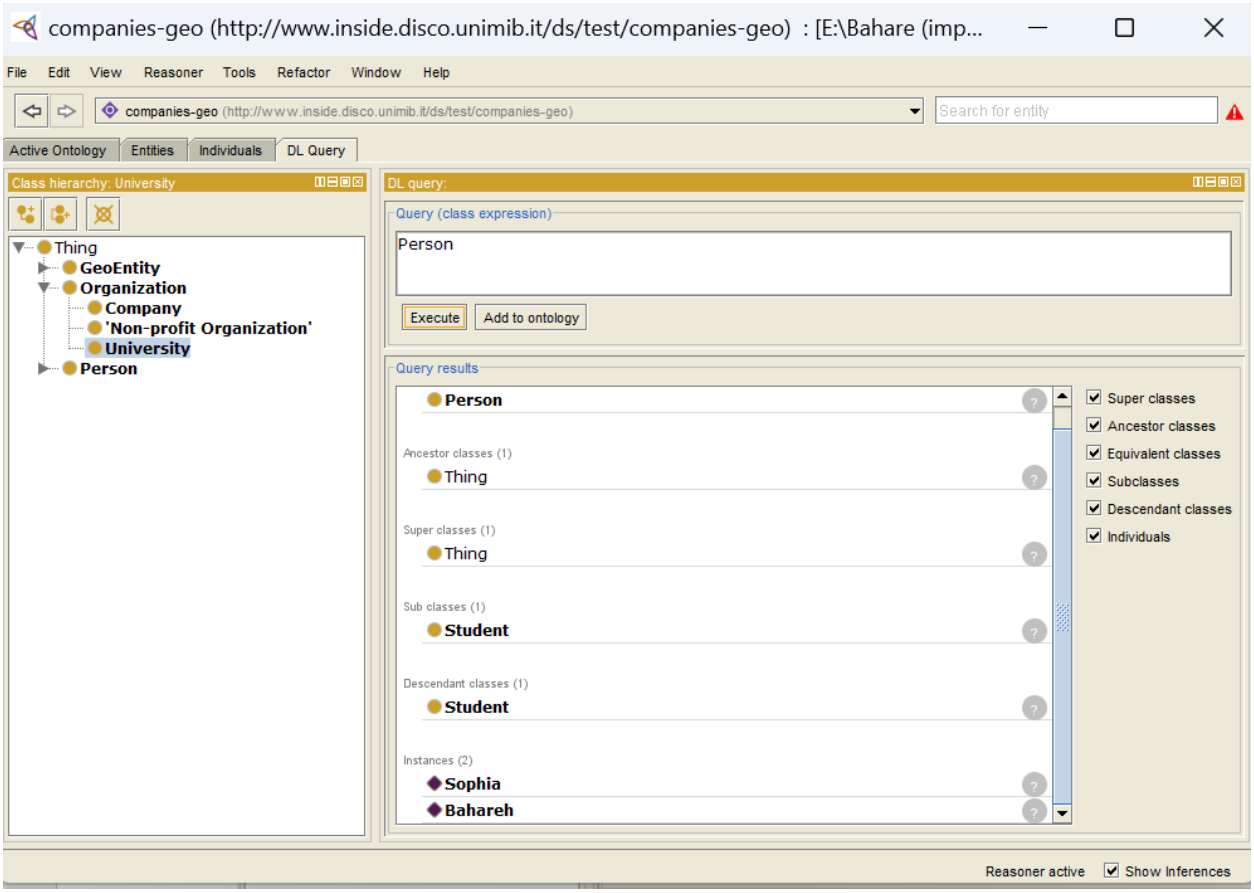
Query 2

```
Person
```

Result:

- Bahareh
- Sophia

This query demonstrated subclass inference.

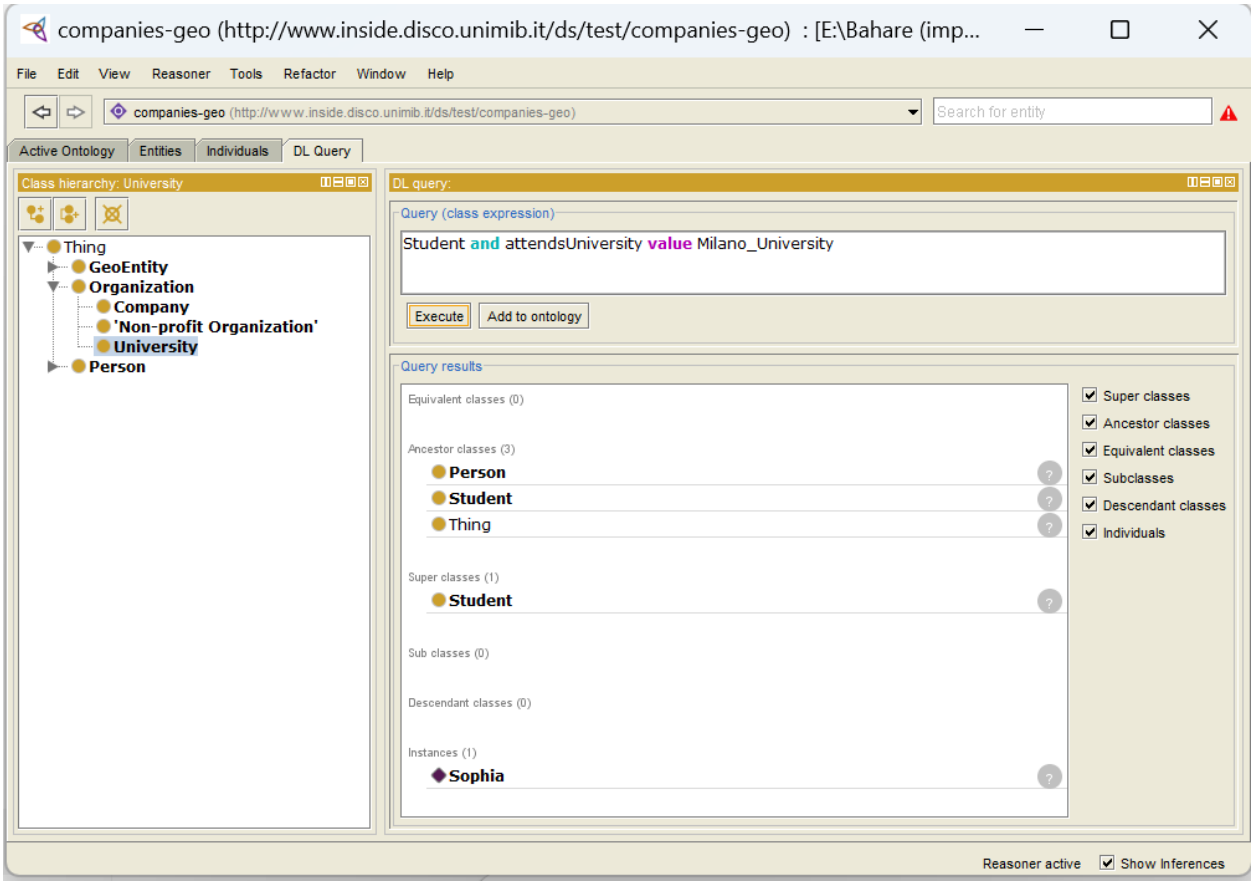


Query 3

```
Student and attendsUniversity value Milano_University
```

Result:

The student who attends the Milano University.



Some improvements :

Additional Ontology Improvements and Semantic Enhancements

To further improve the ontology and increase its semantic expressiveness, several advanced ontology modeling features were implemented. These enhancements strengthened the reasoning capabilities of the ontology and improved semantic validation.

---

1. Inverse Property Definition

An inverse property relationship was introduced between the object properties:

`attendsUniversity`

and

`hasStudent`

The `hasStudent` property was created with:

- Domain: `University`
- Range: `Student`

The property was then configured as the inverse of `attendsUniversity`.

This semantic definition enabled the ontology reasoner to automatically infer reverse relationships. For example, after explicitly inserting:

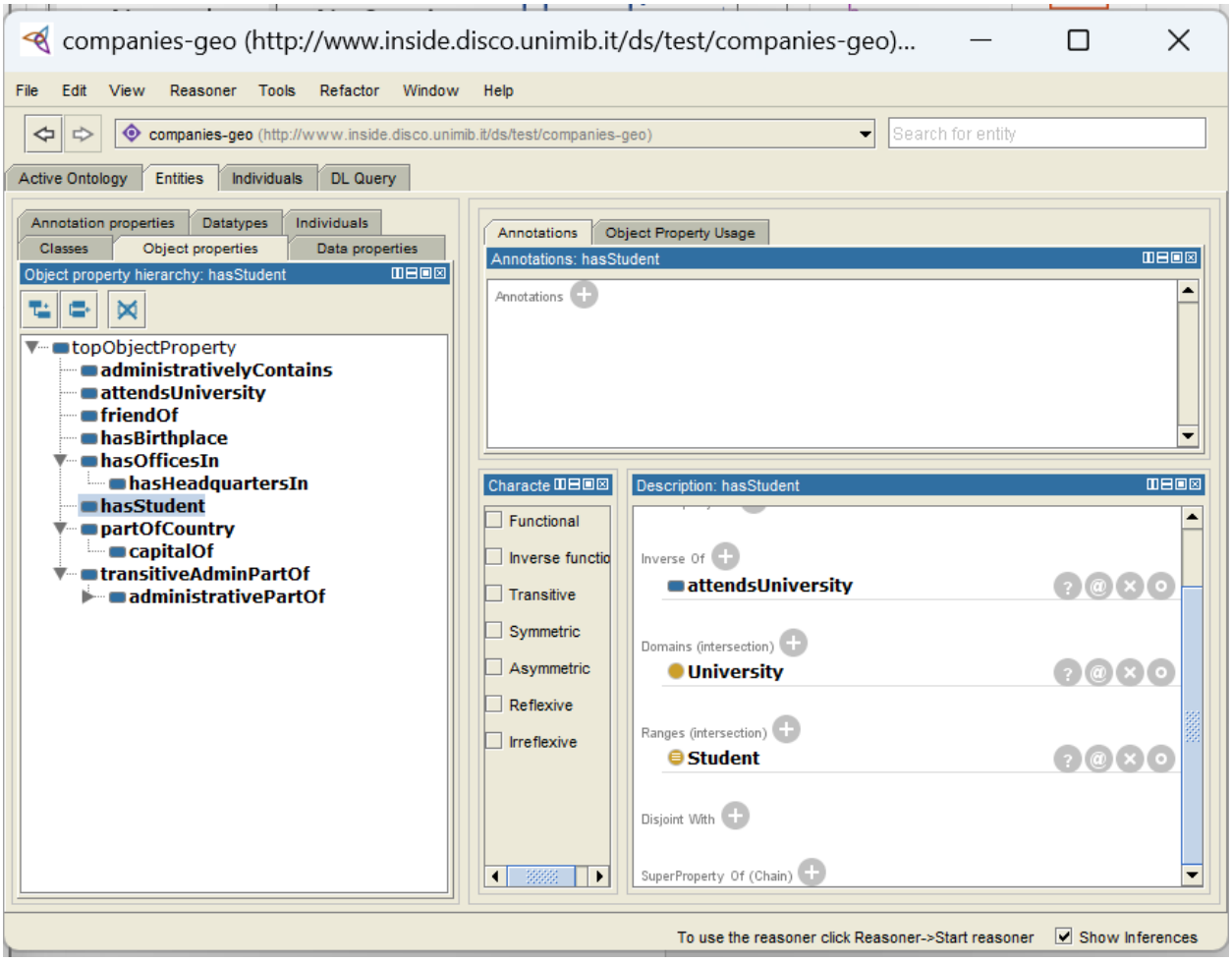
```
Bahareh attendsUniversity Milano_University
```

the reasoner automatically inferred:

```
Milano_University hasStudent Bahareh
```

without manually adding the relation.

This improvement demonstrated successful inverse-property reasoning and automatic semantic inference.



---

2. Functional Property Restriction

The object property:

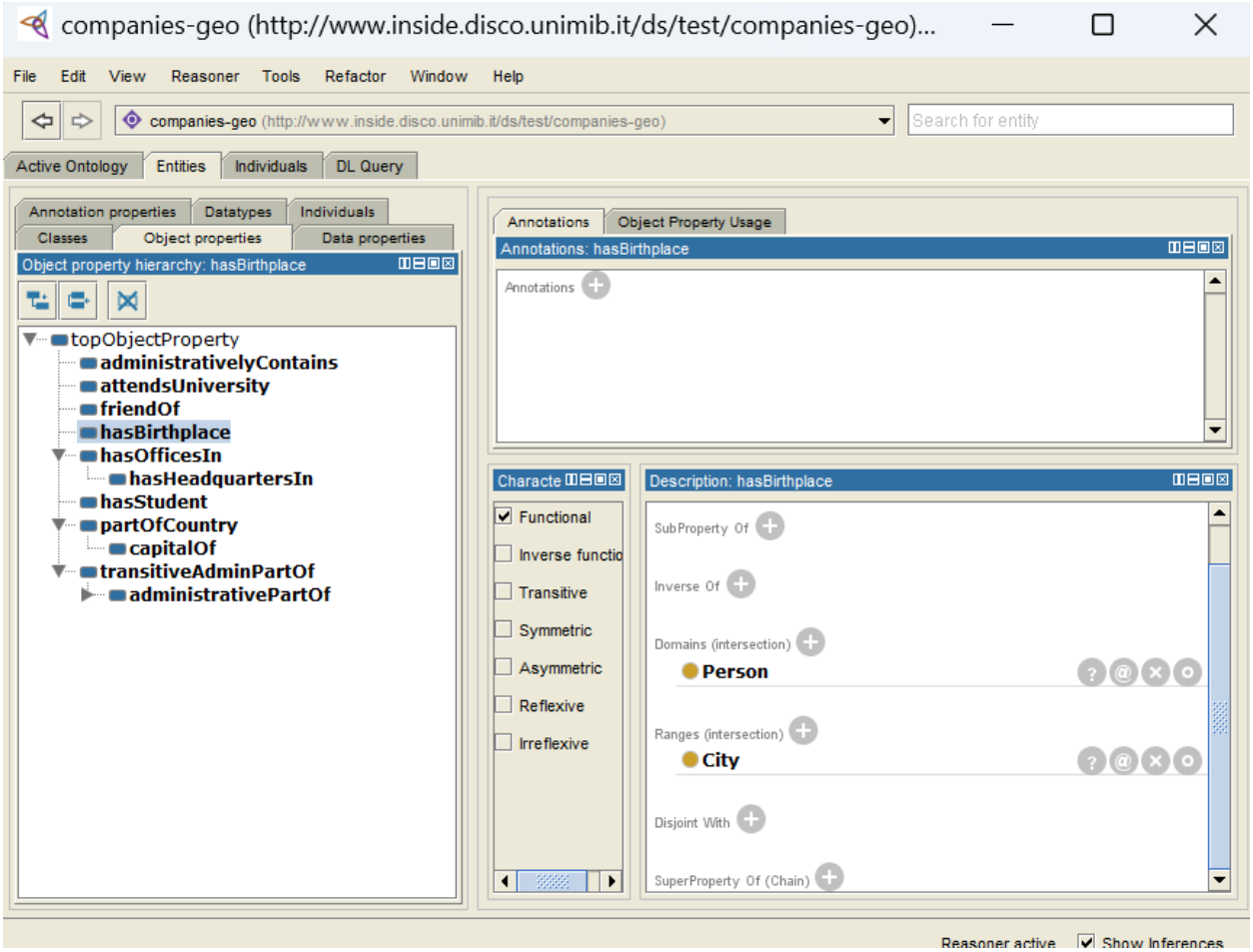
`hasBirthplace`

was configured as a Functional Property.

This semantic restriction ensures that each person can have only one birthplace. The ontology reasoner uses this constraint to detect inconsistent or contradictory assertions.

For example, assigning multiple birthplaces to the same individual would create a logical inconsistency within the ontology.

This enhancement improved semantic validation and ontology consistency checking.



3. Disjoint Class Constraints

Disjoint class restrictions were added between:

`Student`

and

`University`

This semantic constraint ensures that no individual can simultaneously belong to both classes.

For example, if an address is assigned to Bahareh using the hasAddress property, the ontology should not interpret Bahareh as both a Student and a University. Since the domain of hasAddress is University, the disjoint constraint allows the reasoner to detect this contradiction and report an ontology inconsistency instead of accepting an incorrect classification.

Because `Student` and `University` were defined as disjoint classes, the reasoner successfully detected the contradiction and reported an ontology inconsistency.

This demonstrated the effectiveness of semantic constraints and logical validation in ontology engineering.

